

Alzikrayat Photo Sharing Application

Architecture & Implementation Report

Name: Saja Abdalhameed, Information Technology

- **Project Overview:**

Alzikrayat is a photo-sharing web application built from scratch in PHP without any backend framework, following a custom Model-View-Controller (MVC) architecture layered over a 3-tier physical separation (Presentation, Application/Business, Data). The system allows registered users to upload photos, view a shared gallery, comment on photos, and manage their own uploads, including a custom server-side image filter engine (grayscale/sepia) implemented with the PHP GD library.

- **Technology Summary :**

Aspect	Implementation
Language	PHP 8.x (Pure / Standard Library)
Architecture	Custom MVC + manual 3-tier separation (Views / Controllers-Models / Data), Singleton pattern for the DB connection
Routing	Hand-written Router class using regex conversion of {param} placeholders
Database	MySQL via PDO, raw prepared statements, no ORM

Auth	PHP Sessions + bcrypt password_hash(), 7- day last-login cookie
Frontend	HTML5, CSS3, JavaScript (ES6), Bootstrap 5
Security	Prepared statements (SQLi defense), htmlspecialchars() on all output (XSS defense)

- **MVC & 3-Tier Architecture:**

The physical 3-tier layers map onto the logical MVC components as follows:

1. Presentation Tier → Views: PHP templates under views/ (auth, photos, layout) rendered with Bootstrap 5 for responsive layout.
2. Application Tier → Controllers & core logic: AuthController, PhotoController, CommentController, HomeController extend an abstract base Controller that provides render() and redirectTo() helpers.
3. Data Tier → Models: User, Photo, Comment classes extend a base Model that holds the shared PDO connection (Singleton pattern); every query is a parameterized prepared statement.

- **Manual Routing Engine:**

Routing is handled by a custom Router class (core/Router.php) instead of any framework router. Routes are registered as (HTTP verb, path pattern, [Controller, method]) tuples in public/

index.php. On each request, the Router converts each {param} placeholder in the path pattern into a regex capture group ([^/]+), builds a full anchored pattern, and matches it against the incoming URL. Extracted parameters are passed positionally into the target controller method via call_user_func_array. This satisfies the requirement of manual, regex-based parameter capture (e.g. /photo/{id}) without relying on any framework's routing layer.

- **Database Design:**

The MySQL schema consists of three normalized tables: users, photos, and comments. Both photos and comments carry foreign keys (user_id, and photo_id/user_id respectively) with ON DELETE CASCADE, so removing a user automatically removes their photos and comments at the database level. All

primary keys are auto-incrementing integers. Every query in the Model layer uses PDO prepared statements with named or positional placeholders, eliminating raw string concatenation and protecting against SQL injection.

- **Authentication & Session Security:**

User registration validates required fields and hashes passwords using PHP's `password_hash()` (bcrypt) before storing them; plaintext passwords are never persisted. Login establishes a PHP session (`$_SESSION['user_id']`, `$_SESSION['first_name']`) that drives the adaptive navbar ("Hi <name>" + Logout vs. "Please Login"). Independently of the session, a browser cookie records the timestamp of the user's last successful login

with a 7-day expiry, and is surfaced back to the user on the login page as a contextual reminder.

- **Photo Lifecycle: Upload, Display, and Deletion :**

Upload (`store()`) validates the title and file extension (jpg/jpeg/png/gif), generates a unique filename to prevent collisions, moves the file into `public/images/uploads/` via `move_uploaded_file()`, and inserts the metadata row only after the physical write succeeds.

Deletion (`delete()`) is ownership-checked at the controller level (`matchedPhoto['user_id']` must equal the session user) before any destructive action runs. It performs the physical file removal (`unlink()`) on both the working image and any saved filter backup, and only then deletes the database record via a parameterized DELETE ...

WHERE id = ? AND user_id = ? query, giving a second, database-level ownership guarantee.

- **Novelty Feature: Server-Side Image Filters:**

As the project's novelty/creativity component, PhotoController implements GD-based grayscale and sepia filters (applyFilter()). Before the first filter is ever applied to a photo, an untouched copy is saved as a "_original" backup file; a restoreOriginal() action lets the owner revert to that backup at any time. This required no external image-processing API, only PHP's built-in GD extension.

- **Validation & XSS Protection :**

Input is validated on three layers as required by the specification: HTML5 attributes (required, accept) on forms, client-side JavaScript checks

before submission, and server-side PHP validation inside each controller action (e.g. checking file extension whitelist, non-empty title, session ownership). All dynamic output rendered into views is passed through `htmlspecialchars()` to neutralize any injected HTML/JS, mitigating stored and reflected XSS.

- **Deployment :**

The application runs locally under XAMPP/ WAMP with PHP 8.x and MySQL. The alzikrayat folder is placed under htdocs, the alzikrayat database is created and the alzikrayat.sql schema imported via phpMyAdmin, and the app is accessed at <http://localhost/alzikrayat/public/>.

- **Conclusion :**

Alzikrayat satisfies the project's architectural constraints: a hand-written MVC framework, a regex-based manual router, raw parameterized SQL with no ORM, bcrypt-hashed sessions with a persistent last-login cookie, ownership-validated CRUD on photos, and a novelty image-filter feature layered on top of the core requirements.

